



Instituto Tecnológico de Costa Rica

Escuela de Computación

Compiladores e Intérpretes

Grupo: 1

Proyecto - GenLen

Parte 1: Gramática

Allan Bolaños Barrientos - 2024145458

Brian Ramírez Arias - 2024109557

Alejandro Umaña Miranda - 2024130345

Daniel Zeas - 2023147474

Santiago Valverde Álvarez - 2024115742

Prof. Aurelio Sanabria

Miércoles 4 de marzo del 2026

[Introducción](#)

[Motivación](#)

[Nombre del Lenguaje](#)

[Temática y Metáfora](#)

[Característica Única](#)

[Gramática:](#)

[Regla principal:](#)

[Estructura general:](#)

[Procesos:](#)

[Declaraciones](#)

[Asignacion](#)

[Llamadas](#)

[Invocacion Protocolos](#)

[Funciones](#)

[Expresiones](#)

[Literal](#)

[Componentes Léxicos](#)

[Ejemplos](#)

Introducción

Motivación

La bioinformática y la biología computacional han experimentado un crecimiento exponencial en la última década, impulsadas por avances como la secuenciación masiva de ADN, la predicción de estructuras proteicas mediante inteligencia artificial (AlphaFold) y la edición genética con sistemas CRISPR-Cas9. Sin embargo, los lenguajes de programación tradicionales no están diseñados específicamente para manejar las abstracciones naturales de este dominio.

GenLen surge como respuesta a la necesidad de un lenguaje de propósito específico que permita a investigadores, bioinformáticos y estudiantes trabajar con conceptos biológicos de forma más intuitiva y expresiva. En lugar de manipular únicamente cadenas de texto o números, GenLen permite trabajar directamente con secuencias genéticas como ciudadanos de primera clase del lenguaje, usando una sintaxis que refleja las operaciones reales de laboratorio.

Nombre del Lenguaje

GenLen (de *Genetic Language*)

El nombre refleja el enfoque central en genética y secuencias genómicas, siendo a la vez corto, memorable y autodescriptivo.

Temática y Metáfora

GenLen utiliza la bioinformática y la ingeniería genética como metáfora central de su diseño. Cuando se programa en GenLen, no se siente como un desarrollador tradicional escribiendo código abstracto, sino como un bioingeniero trabajando en un laboratorio: diseñando experimentos, analizando secuencias de ADN y manipulando material genético, pero todo de forma virtual. Esta idea del "laboratorio de programación" no es solo un nombre bonito, sino que está presente en todo el lenguaje: en cómo se llaman las cosas, en cómo se escriben las instrucciones, y en cómo piensas sobre lo que estás haciendo cuando programas.

Característica Única

La característica distintiva de GenLen radica en su sistema de tipos genéticos nativos, el cual eleva las secuencias biológicas de simples cadenas de texto a entidades de primera clase dentro del lenguaje. Mientras que en lenguajes tradicionales como Python o Java una secuencia de ADN es simplemente un string ("ATGCGTA"), en GenLen estas secuencias son tipos de datos fundamentales con comportamientos y operaciones específicas incorporadas al lenguaje mismo.

GenLen introduce tres tipos genéticos fundamentales: Gen (que representa secuencias de ADN con nucleótidos A, G, C, T), ARN (secuencias de ARN mensajero con nucleótidos A, G, C, U), y Proteína (cadenas de aminoácidos). Estos tipos no son meros alias o clases definidas por el usuario, sino tipos primitivos del lenguaje con sintaxis especial: las secuencias genéticas se escriben entre comillas simples y terminan con un punto, por ejemplo: Gen secuencia = 'ATGGCCTAGCTA.', diferenciándose visualmente de las cadenas de texto convencionales.

Lo verdaderamente único es que estos tipos genéticos vienen equipados con operaciones moleculares nativas que reflejan procesos biológicos reales. Un Gen puede .transcribir() para convertirse en ARN, simulando el proceso celular de transcripción. Un ARN puede .traducir() para generar una Proteína siguiendo las reglas del código genético. Además, GenLen incorpora operaciones avanzadas como .cas9() para edición genética tipo CRISPR, .fold3D() para predicción de estructuras tridimensionales inspirada en AlphaFold, .dock() para simulación de acoplamiento molecular, .mutar() para introducir mutaciones puntuales, y .alinear() para comparación de secuencias. Estas operaciones no son funciones de biblioteca externa, sino métodos intrínsecos del lenguaje, accesibles directamente desde los tipos genéticos.

Esta integración profunda de la biología molecular en la sintaxis y semántica del lenguaje permite que el código se lea como un protocolo de laboratorio real, reduciendo la brecha cognitiva entre el pensamiento biológico y la implementación computacional. Un bioinformático puede escribir `proteina_final = gen_original.transcribir().traducir()` y ver reflejada exactamente la cadena dogma central de la biología molecular (ADN → ARN → Proteína) de forma directa y natural, sin necesidad de importar bibliotecas especializadas o manejar conversiones manuales entre representaciones.

Nombre : GenLen

Temática: GenLen permite al programador actuar como un bioingeniero que diseña y manipula secuencias de ADN, ARN y proteínas. Las instrucciones del lenguaje representan operaciones de laboratorio como transcripción, traducción, mutación y recombinación genética. La metáfora central es que el código es un "experimento" y las variables son "tubos de ensayo" que contienen material genético.

Característica única:

Gen 'AGGTGCGTAGGACGGTGAGAGGCGTATGAGGAGCTATA.'

ARN

AMI

PRO: Ejecutan una función ->>> helloworld

gen.mutate()

gen.dock('Gen aaaaaa')

gen.

Ejemplos:

2 Tipos de datos:

- **Gen:** Representa una cadena de ADN (Alpha)
- **Númericos:** números

3 En lenguaje recibe cadenas de ADN y se permite interactuar con ellas.

4. Lenguaje:

- **Cas9 *****
- **Mutaciones**
- **Docking (Por medio de un solver) *****
- **Geometrica 3D (Alphafold)**
- **Transcripción**
- **Proteica**
- **Reducida**
- **Alineamiento de secuencia**

Gramática:

Regla principal:

PROGRAMA ::= LABORATORIO

Estructura general:

LABORATORIO ::= "lab" IDENTIFICADOR BLOQUE

BLOQUE ::= "/"\ " LISTA_PROCESOS "\ "

LISTA_PROCESOS ::= (PROCESO LISTA_PROCESOS)?

Procesos:

PROCESO ::= DECLARACION
| ASIGNACION
| (LLAMADA | INVOCACION_PROTOCOLO) ";"
| PROTOCOLO
| BLOQUE

Declaraciones

DECLARACION ::= "muestra" IDENTIFICADOR ":" TIPO ("=" EXPRESION)?
";"

TIPO ::= "adn" | "arn" | "prot" | "num"

Asignacion

ASIGNACION ::= IDENTIFICADOR "=" EXPRESION ";"

Llamadas

LLAMADA ::= IDENTIFICADOR "->" MECANISMO "(" ARGUMENTOS ")"

MECANISMO ::= "mutar"
 | "transcribir"
 | "traducir"
 | "docking"
 | "cas9"
 | "alinear"
 | "modelar"

ARGUMENTOS ::= (LISTA_ARGUMENTOS)?

LISTA_ARGUMENTOS ::= EXPRESION RESTO_ARGUMENTOS

RESTO_ARGUMENTOS ::= ", " (EXPRESION RESTO_ARGUMENTOS)?

Invocacion Protocolos

INVOCACION_PROTOCOLO ::= IDENTIFICADOR "(" ARGUMENTOS ")"

Funciones

PROTOCOLO ::= "protocolo" IDENTIFICADOR "(" PARAMETROS ")" BLOQUE

PARAMETROS ::= (LISTA_PARAMETROS)?

LISTA_PARAMETROS ::= PARAMETRO RESTO_PARAMETROS

RESTO_PARAMETROS ::= ", " (PARAMETRO RESTO_PARAMETROS)?

PARAMETRO ::= IDENTIFICADOR ":" TIPO

Expresiones

EXPRESION ::= EXP_LOGICA

EXP_LOGICA ::= EXP_COMP RESTO_LOGICA

```

RESTO_LOGICA ::= ("y" | "o") (EXP_COMP RESTO_LOGICA)?

EXP_COMP ::= EXP_ARIT RESTO_COMP

RESTO_COMP ::= ("==" | "!=" | "<" | "<=" | ">" | ">=") EXP_ARIT
RESTO_COMP)?

EXP_ARIT ::= TERMINO RESTO_ARIT

RESTO_ARIT ::= ("+" | "-") TERMINO RESTO_ARIT)?

TERMINO ::= FACTOR RESTO_TERMINO

RESTO_TERMINO ::= ("*" | "/" | "%") FACTOR RESTO_TERMINO)?

FACTOR ::= ("- " | "+") FACTOR | PRIMARIO

LLAMADA ::= IDENTIFICADOR "->" MECANISMO "(" ARGUMENTOS ")"

MECANISMO ::= "mutar"
              | "transcribir"
              | "traducir"
              | "docking"
              | "cas9"
              | "alineal"
              | "modelar"

```

Literal

```

LITERAL ::= LIT_ADN
           | LIT_ARN
           | LIT_PROT
           | NUMERO
           | BOOLEANO

LIT_ADN ::= "adn" CADENA_ADN

LIT_ARN ::= "arn" CADENA_ARN

LIT_PROT ::= "prot" CADENA_PROTEINA

BOOLEANO ::= "V" | "F"

```

Componentes Léxicos

IDENTIFICADOR ::= [a-zA-Z_][a-zA-Z0-9_]*

NUMERO ::= [0-9]+(\.[0-9]+)?

CADENA_ADN ::= "'" [AGCT]+ "'"

CADENA_ARN ::= "'" [AUCG]+ "'"

CADENA_PROTEINA ::= "'" [ACDEFGHIKLMNPQRSTVWY]+ "'"

Ejemplos

```
// Mini demo
lab MiniLab /\

    muestra dnaA : adn = adn 'ATGCGT';
    muestra dnaB : adn = adn 'ATGAGT';

    protocolo comparar(a: adn, b: adn) /\
        a->align(b, 0.90);
        a->mutate(1);
    \ /

    comparar(dnaA, dnaB);

\ /
```

Ejemplo 1: declarar y transcribir una secuencia

```
lab Ejemplo1 /\
    muestra gen1 : adn = adn 'ATGCGT';
    muestra arn1 : arn;
    arn1 = gen1->transcribe();
\ /
```

Explicación:

lab Ejemplo1 /\	Abre el programa, se llama Ejemplo1
muestra gen1 : adn = adn 'ATGCGT';	Declara una variable llamada gen1 de tipo adn y le asigna la secuencia ATGCGT. Cumple la regla DECLARACION con el = EXPRESION opcional incluido. La expresión es adn 'ATGCGT' que es un LIT_ADN .

muestra arn1 : arn;	Declara una variable arn1 de tipo arn sin valor inicial, lo cual es válido porque el = EXPRESION es opcional en DECLARACION .
arn1 = gen1->transcribe() ;	Esto es una ASIGNACION . El lado derecho es una LLAMADA: gen1->transcribe() . transcribe está en la lista de MECANISMO y no necesita argumentos, entonces ARGUMENTOS queda vacío, lo cual es válido porque ARGUMENTOS ::= (LISTA ARGUMENTOS)? es opcional. El resultado se guarda en arn1.
\\	cierra el bloque del lab.

Ejemplo 2: comparar dos secuencias

```
lab Ejemplo2 /\
    muestra secA : adn = adn 'ATGCGT';
    muestra secB : adn = adn 'ATGAGT';
    secA->align(secB);
\\
```

Explicación:

Las dos primeras líneas son iguales al ejemplo anterior, dos variables adn con sus secuencias.	
secA->align(secB);	Esto es una LLAMADA usada como PROCESO directamente (no dentro de una asignación). La gramática lo permite: PROCESO ::= (LLAMADA INVOCACION PROTOCOLO) ";" . El método align está en MECANISMO. El argumento secB es un IDENTIFICADOR , que es un PRIMARIO , que es un FACTOR , que es un TERMINO ... y así sube por la cadena de expresiones hasta ser un EXPRESION válido en LISTA ARGUMENTOS .

Ejemplo 3: protocolo que edita y traduce una secuencia

```
lab Ejemplo3 /\
    muestra genBase : adn = adn 'ATGGCCTAGCTA';
    muestra resultado : prot;

    protocolo editarYTraducir(g: adn, corte: adn) /\
```

```

    muestra editado : adn;
    muestra mensajero : arn;
    editado = g->cas9(corte);
    mensajero = editado->transcribe();
    mensajero->translate();
  \

  resultado = editarYTraducir(genBase, adn 'TAGCTA');
\

```

muestra genBase : adn = adn 'ATGGCCTAGCTA';	Variable adn con secuencia inicial.
muestra resultado : prot;	Variable de tipo prot sin valor inicial, donde vamos a guardar el resultado final.
protocolo editarYTraducir(g: adn, corte: adn) /\n... \n	Define una función. Cumple la regla PROTOCOLO . Tiene dos parámetros (g y corte), ambos de tipo adn . Dos parámetros separados por coma es válido según LISTA_PARAMETROS y RESTO_PARAMETROS .
muestra editado : adn; muestra mensajero : arn;	Variables locales dentro del protocolo. La gramática no distingue explícitamente locales de globales pero estas están dentro del bloque del protocolo así que conceptualmente son locales.
editado = g->cas9(corte);	Asignación. Llama cas9 sobre g pasando corte como argumento. cas9 está en MECANISMO . Válido. mensajero = editado->transcribe(); - asignación. Transcribe el ADN editado a ARN. Válido.
mensajero = editado->transcribe();	Asignación. Transcribe el ADN editado a ARN. Válido.
mensajero->translate ());	Llama translate sobre el ARN. Esta línea es una LLAMADA como proceso directo, igual que en el ejemplo 2. Nótese que el resultado de translate no se está guardando en ninguna

	variable aquí, lo cual es válido porque PROCESO permite una LLAMADA sola.
<pre> resultado = editarYTraducir(genB ase, adn 'TAGCTA'); </pre>	Esto es una ASIGNACION. El lado derecho es una INVOCACION PROTOCOLO: el nombre del protocolo seguido de argumentos entre paréntesis. Los argumentos son genBase (un identificador) y adn 'TAGCTA' (un literal), separados por coma. Válido según LISTA_ARGUMENTOS.

Ejemplo 4: protocolo que transcribe y traduce

```

lab Ejemplo4 /\
  muestra resultado : prot;

  protocolo procesar(g: adn) /\
    muestra rna : arn;
    rna = g->transcribe();
    rna->translate();
  \/

  resultado = procesar(adn 'ATGCGT');

\/

```

<pre> protocolo procesar(g: adn) /\ </pre>	Define un protocolo. Cumple la regla: PROTOCOLO ::= "protocolo" IDENTIFICADOR "(" PARAMETROS ")" BLOQUE Tiene un parámetro g de tipo adn, válido según PARAMETRO ::= IDENTIFICADOR ":" TIPO.
<pre> muestra rna : arn; </pre>	Variable local de tipo arn dentro del bloque del protocolo.
<pre> rna = g->transcribe(); </pre>	Es una ASIGNACION. El lado derecho es una LLAMADA.
<pre> resultado = procesar(adn 'ATGCGT'); </pre>	Es una ASIGNACION. El argumento adn 'ATGCGT' es un LIT_ADN.

Ejemplo 5: mutar y comparar

```
lab ejemplo5 /\n  muestra genOriginal : adn = adn 'ATGCGT';\n  muestra genMutado : adn = adn 'ATGCGT';\n\n  genMutado->mutate(1);\n  genOriginal -> align(genMutado, 0.90);
```